

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №3
по дисциплине: БД
тема: «Нормализация отношений в базе данных»

Выполнил: студент группы ВТ-231
Масленников Д. А.
Проверили:
Панченко М. В.

Белгород 2025

Цель работы: изучить нормальные формы отношений, получить навыки последовательной нормализации отношений.

Задание к работе:

1. Нормализовать структуру базы данных, разработанную в предыдущих лабораторных работах. Доказать соответствие 3 NF, выписав функциональные зависимости для каждой сущности.
2. Привести данное в задании отношение к 3 NF согласно номеру варианта.

Вариант 2

ФИО студента	Адрес	Код курса	Название курса	Длительность	Стоимость
Смирнов А. А.	Новая 1-13	102	Web-дизайн	40	2600
Егунев Е. А.	Полевая 34-2	134	Компьютерная графика	56	3500
Ганич И. И.	Майская 2-3	102	Web-дизайн	40	2600
Авдеева Г. В.	Новая 1-18	156	HTML	36	1500
Бобряк Д. Ю.	Садовая 3-78	107	MS Access	24	2000
Саковец М. О.	Лесная 24-2	134	Компьютерная графика	56	3500
Ларин М. Р.	Полевая 32-6	104	ООП	56	5000
Иванов И. А.	Лесная 3-56	102	Web-дизайн	40	2600
Ковалев Р. А.	Красная 4-12	102	Web-дизайн	40	2600
Шепелев С. А.	Новая 3-23	107	MS Access	24	2000
Заяц А. А.	Новая 5-12	104	ООП	56	5000
Иванова Е. Е.	Красная 2-3	156	HTML	36	1500

Задание 1:

В моей предыдущей работе структура и так соответствовала 3NF. Ее структура:

```

railway=> \dt

```

Список отношений			
Схема	Имя	Тип	Владелец
public	car	таблица	railuser
public	contactdetails	таблица	railuser
public	passenger	таблица	railuser
public	route	таблица	railuser
public	schedule	таблица	railuser
public	seat	таблица	railuser
public	ticket	таблица	railuser
public	ticketstatus	таблица	railuser
public	train	таблица	railuser

(9 строк)

id		fullname	phone	email	passportdata		
cb41424f-e336-4988-8ef2-53107d5ebb84		Contact User Number2	+791231231231	contact2@gmail.com	1231	123123	
d2715033-c4ae-4526-aaf2-f961b2c76fb1		Contact User Number3	+791111111111	contact3@yandex.ru	1234	123412	
5fde1071-3c3b-4def-9314-34309b122ca7		Contact User Number4	+797777777777	contact4@gmail.com	1111	111111	
aa066b4a-fb13-442a-adab-ef497542cf80		Contact User Number1	+794578238841	updated1@example.com	1231	231231	
(4 строки)							
id		contactdetailsid					
54848202-ad69-4a0e-a5d5-909e7c96ac2b		cb41424f-e336-4988-8ef2-53107d5ebb84					
1022a58d-e47c-48d7-9cbd-14366105ecd2		d2715033-c4ae-4526-aaf2-f961b2c76fb1					
e522cddf-d016-4bec-8a4e-ad522f655d04		5fde1071-3c3b-4def-9314-34309b122ca7					
12fd0995-657b-4900-8aa2-46c6936b2061		aa066b4a-fb13-442a-adab-ef497542cf80					
(4 строки)							
id		passengerid		seatid	statusid	saledatetime	price
cb5d1bc2-66cb-44a9-8321-1792acd14a50		54848202-ad69-4a0e-a5d5-909e7c96ac2b		2	1	2025-09-24 16:46:39.992381	3200.00
c090d5415-c953-4888-b147-31b23e4a6b67		1022a58d-e47c-48d7-9cbd-14366105ecd2		3	3	2025-09-24 16:46:39.999171	2800.00
b899ae43-3dbe-4404-8796-6775bdf64318		e522cddf-d016-4bec-8a4e-ad522f655d04		4	2	2025-09-24 16:46:40.001806	1500.00
1717290b-0353-4ae5-808f-a27d797ae068		54848202-ad69-4a0e-a5d5-909e7c96ac2b		2	1	2025-09-24 16:46:45.083637	3200.00
552424e0-1768-49da-996c-90b3dc228d84		1022a58d-e47c-48d7-9cbd-14366105ecd2		3	3	2025-09-24 16:46:45.086869	2800.00
4aa0f471-306f-47df-a7bf-5b8432489e91		e522cddf-d016-4bec-8a4e-ad522f655d04		4	2	2025-09-24 16:46:45.090092	1500.00
920c4ab4-d85d-45cd-97b8-40898c5a7988		12fd0995-657b-4900-8aa2-46c6936b2061		1	1	2025-09-24 16:46:45.076524	4700.00
(7 строк)							
id	trainnumber	traintype					
2	002B	пассажирский					
3	003B	фирменный					
4	004Г	скорый					
1	001A	фирменный					
(4 строки)							

id	departurestation	arrivalstation	distance	durationminutes
2	Москва	Казань	800	540
3	Санкт-Петербург	Новгород	200	120
4	Казань	Екатеринбург	900	600
1	Москва	Санкт-Петербург	700	270

(4 строки)

id	trainid	routeid	departuredatetime	arrivaldatetime
2	2	2	2025-06-02 09:00:00	2025-06-02 18:00:00
3	3	3	2025-06-03 10:00:00	2025-06-03 12:00:00
4	4	4	2025-06-04 11:00:00	2025-06-04 21:00:00
1	1	1	2025-06-01 09:00:00	2025-06-01 12:30:00

(4 строки)

id	scheduleid	carnumber	cartype	totalseats
2	1	2	плацкарт	54
3	2	1	сидячий	81
4	3	1	СВ	18
1	1	1	купе	40

(4 строки)

id	carid	seatnumber	seattype
2	1	2	лежачее
3	2	1	лежачее
4	3	1	сидячее
5	4	1	лежачее
1	1	1	сидячее

(5 строк)

id	statusname
1	активен
3	забронирован
2	возвращен

(3 строки)

Доказательство соответствия 3NF:

Отношение находится в 3NF, если:

1. Оно в 1NF (все атрибуты атомарны),
2. В 2NF (неключевые атрибуты полностью зависят от всего РК),
3. Нет транзитивных зависимостей (неключевой → неключевой).

Рассмотрим каждую таблицу моей БД

ContactDetails

- PK: ID
- FD: ID → FullName, Phone, Email, PassportData
- Все атрибуты зависят напрямую от РК, транзитивных зависимостей нет → **3NF**

Passenger

- PK: ID
- FD: ID → ContactDetailsID
- Один неключевой атрибут, зависит от РК → 3NF

Train

- PK: ID
- FD: ID → TrainNumber, TrainType

- Нет зависимости между TrainNumber и TrainType → 3NF

Route

- PK: ID
- FD: ID → DepartureStation, ArrivalStation, Distance, DurationMinutes
- Все атрибуты зависят от PK; даже если (Departure, Arrival) → Distance, это не нарушает 3NF при PK = ID → 3NF

Schedule

- PK: ID
- FD: ID → TrainID, RouteID, DepartureDateTime, ArrivalDateTime
- Все атрибуты зависят от PK → 3NF

Car

- PK: ID
- FD: ID → ScheduleID, CarNumber, CarType, TotalSeats
- Нет зависимости, например, CarType → TotalSeats → 3NF

Seat

- PK: ID
- FD: ID → CarID, SeatNumber, SeatType
- Уникальность (CarID, SeatNumber) не нарушает 3NF → 3NF

TicketStatus

- PK: ID
- FD: ID → StatusName
- Тривиальный случай → 3NF

Ticket

- PK: ID
- FD: ID → PassengerID, SeatID, StatusID, SaleDateTime, Price
- Price хранится как атрибут билета (не выводится из других полей), поэтому нет транзитивной зависимости → 3NF

Задание 2:

ФИО студента	Адрес	Код курса	Название курса	Длительность	Стоимость
Смирнов А. А.	Новая 1-13	102	Web-дизайн	40	2600
Егунев Е. А.	Полевая 34-2	134	Компьютерная графика	56	3500
Ганич И. И.	Майская 2-3	102	Web-дизайн	40	2600
Авдеева Г. В.	Новая 1-18	156	HTML	36	1500
Бобряк Д. Ю.	Садовая 3-78	107	MS Access	24	2000
Саковец М. О.	Лесная 24-2	134	Компьютерная графика	56	3500
Ларин М. Р.	Полевая 32-6	104	ООП	56	5000
Иванов И. А.	Лесная 3-56	102	Web-дизайн	40	2600
Ковалев Р. А.	Красная 4-12	102	Web-дизайн	40	2600
Шепелев С. А.	Новая 3-23	107	MS Access	24	2000
Заяц А. А.	Новая 5-12	104	ООП	56	5000
Иванова Е. Е.	Красная 2-3	156	HTML	36	1500

Изначальная таблица обладает несколькими недостатками:

1) Повторяется информация о курсах.

Например, «Web-дизайн» с кодом 102, длительностью 40 и стоимостью 2600 записан столько раз, сколько студентов на него записалось. Если курс подорожает, то нужно менять все строки, где он есть. Звучит совсем логично

2) Нельзя добавить курс, если на него никто не записался.

Хочешь добавить новый курс? Нельзя — потому что нет студента, а значит, нечего вставить в колонку «ФИО студента». Это аномалия вставки.

3) Если последний студент уйдёт с курса — информация о курсе исчезнет.

Удалили последнюю запись про «HTML» — и всё, база «забыла», что такой курс вообще существовал. Это аномалия удаления.

4) Адрес студента повторяется, если он записан на несколько курсов.

(В твоих данных — пока по одному курсу, но в реальности может быть иначе.)

→ При смене адреса — нужно обновлять все его записи.

Переделанный вариант:

Запросы для нового варианта:

```
CREATE TABLE Student (
```

```
    StudentID SERIAL PRIMARY KEY,
```

```
    FullName VARCHAR(100) NOT NULL,
```

```
    Address VARCHAR(200) NOT NULL
```

```
);
```

```
CREATE TABLE Course (
```

```
    CourseID INT PRIMARY KEY,
```

```
    CourseName VARCHAR(100) NOT NULL,
```

```
    Duration INT NOT NULL CHECK (Duration > 0),
```

```
    Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0)
```

```
);
```

```
CREATE TABLE Enrollment (
```

```
    StudentID INT REFERENCES Student(StudentID) ON DELETE CASCADE,
```

```
    CourseID INT REFERENCES Course(CourseID) ON DELETE CASCADE,
```

```
    PRIMARY KEY (StudentID, CourseID)
```

```
);
```

Результат:

```
lab3=# \dt
```

Список отношений			
Схема	Имя	Тип	Владелец
public	course	таблица	postgres
public	enrollment	таблица	postgres
public	student	таблица	postgres

(3 строки)

У нас стало 3 таблицы:

1. **Студенты** — только информация о студентах:

1 Смирнов А. А. Новая 1-13

2 Ганич И. И. Майская 2-3

→ Адрес хранится **один раз**, даже если студент учится на 10 курсах.

2. **Курсы** — только информация о курсах:

102 Web-дизайн 40 2600

134 Компьютерная графика 56 3500

→ Курс существует **независимо от студентов**. Можно добавлять, менять, удалять — без привязки к кому-то.

3. **Зачисления** — кто на что записан:

1 102

2 102

3 134

Доказательство 3NF для нормализованной структуры (вариант 2)

Исходное отношение имело составной ключ (**ФИО, Код курса**) и нарушало 2NF из-за зависимостей от части ключа:

- ФИО → Адрес
- Код курса → Название, Длительность, Стоимость

После декомпозиции получены три таблицы:

1. **Student(StudentID PK, FullName, Address)**

- FD: StudentID → FullName, Address
- Все неключевые атрибуты зависят от PK, транзитивных зависимостей нет → 3NF

2. **Course(CourseID PK, CourseName, Duration, Price)**

- FD: CourseID $\rightarrow$ CourseName, Duration, Price
- Нет зависимостей между неключевыми атрибутами $\rightarrow$ 3NF

3. Enrollment(StudentID, CourseID)

- PK: (StudentID, CourseID)
- Неключевых атрибутов нет $\rightarrow$ 3NF

Все таблицы находятся в **третьей нормальной форме**. Избыточность и аномалии устранены.

Вывод: в ходе лабораторной работы мы изучили 3NF, чтобы строить более гибкие и расширяемые базы данных